

一、shuxuejuan 简介

1.一句话简介：一个用于（中学）简单试卷、练习、讲义、笔记快速制作的 Typst 包。

2.核心目标：

(1)开箱即用，提供题目/答案相关，与其他常用试卷内容的函数；

(2)同时尽可能少地干预题目/答案以外的排版（可选用 `#show: shuxuejuan` 来将常用函数绑定到 Typst 的内置函数上，但这不是必须的）。

3.核心功能，提供：

(1)问题相关：独立计数器，简单够用的题号（计数、对齐）相关设置（见 五、1. 与 五、3.）；

(2)答案相关：答案的统一设置，包括颜色与不影响其他内容排版的可见性设置；

(3)其他：选择题选项排版、对题号的引用、学生信息栏等相关函数。

二、常用函数（使用时）：见 四、4.(1) 。

三、例子（仅供参考，见 六、6. 与 六、4.）

1.作为模板使用（使用 `#show: shuxuejuan`）：../example.typ；

2.作为普通包使用（不使用 `#show: shuxuejuan`）：../example-no-template.typ。

四、代码架构（标蓝者为核心）

1. 基础

(1) env.typ

①counter-question：一种counter-with-acc（见 五、1. ），本文件中的相关函数：

- counter-with-acc相关：counter-with-acc-step, counter-with-acc-get, counter-with-acc-update, sxj-counter-question-update, 与counter中的step, get, update 相对应；
- counter-question专属：sxj-counter-with-acc-to-nums-default与sxj-counter-with-acc-to-nums-normal, 详见 五、2. .

②env相关：

- 一些不常变但常用的（全局）变量，当前¹为各函数提供可覆盖的默认值：
 - font-size：一个至少包含medium、big、huge的字典（当前主要控制sxj-question-zh及sxj-title的字符大小）；
 - qst-style：sxj-question在调用sxj-term时使用默认的composer；
 - fn-number：控制counter-with-acc (counter-question) 至array的转化方式（详见 五、2.(1) ）；
 - qst-tag-w：数组，控制各级题号宽度，元素可为auto, length或包含max或min的字典²；
 - ans-shown：是否显示答案；
 - ans-color：答案颜色；
 - ref-style：sxj-ref-to-question的默认ref-style；
- 相关常用函数：
 - env-get：通过key获取特定的变量值，如env-get("ans-shown")；
 - env-upd：更新env中的相关值，如env-upd(ans-shown: false)；
 - with-env：临时更新env中的值，如with-env(ans-shown: false)[#body]，则body中的env的ans-shown会被临时设置为false, body以外的ans-shown值不变。

③其他

- SXJ-BODY-TYPE：用于为sxj-question与sxj-answer做metadata标记³；
- counter-answer：用以区分一个问题中的多个（处）答案（例如多个空的填空题）。

(2) term.typ

①核心函数：sxj-term

类似标准库中的terms，用于将tag与body打包，详见 五、3. .

②辅助函数：

- sxj-content-trim：去除content中的前导空格，如：

```
#[
  body
]
```

可能为"body"添加不必要的空格。

- sxj-get-composer-for：为body提供用于sxj-term的composer建议，当前仅检测是否含grid.

2. 基础应用

(1) question.typ

①sxj-numbering-numbers：非重点，详见 五、2. ；

②sxj-question

以sxj-term为基础，与counter-question绑定以提供题号，提供metadata绑定³，提供更丰富的hanging-indent设置。

③sxj-question-zh

sxj-question的中文优化版，预设题号对齐及题目字体大小。本包内有关题目设置（如show heading: .., sxj-qg-to-qst-zh等）应优先使用sxj-question-zh而非sxj-question，以确保排版一致。

(2) answer.typ

①sxj-answer

为本包中的答案提供同一接口，以统一管理shown, fill, 附加metadata³。是所有答案相关函数的底层。

②sxj-answer-sol：应用题答案与“解：”或“证明：”，提供高度占位控制；

③sxj-bracket：选择题答案与左右括号，适用于单选与（不超过4字母答案的）多选，答案为多选时左右括号距离不随答案个数变动，防止猜答案个数；

④sxj-blank：填空题答案与“空”，可自动/手动设置空的宽度。

¹指“当前版本”，下同。
²不建议为字典，详见 六、5.(1) .
³当前对于这些metadata我们只标记，不使用（你可用#context query(<sxj-label-answer>)等来重现答案）。

第 2 页 共 5 页

3. 其他应用

(1) `question-group.typ`: 批量处理相同格式的问题与答案, 提供批量处理函数, 常用于为判断题、初中小学的计算题等排版.

① `sxj-question-group`:

- 批量处理问题与答案的核心函数, 核心:
`let cnt = batcher.fold(contents.pos(), (acc, pcs) => pcs(envs, acc))`
- 核心参数 `batcher`: 用于批量处理的函数数组, 里面每个函数均是纯函数, 接受 `envs` (可能用到的环境变量, 非 `env.typ` 中的 `env`) 与 `cnts` (需处理到的内容) 作为参数, 返回处理后的内容.
- `cnts` (`contents`) 的大致处理流程:
 - 预处理: 确保 `cnts` 是问题答案相间的: 根据需要调用 `sxj-qg-ins-ans-empty`;
 - 正式处理: 根据需要调用批量处理函数, 有了上一步的处理, 此处的函数结构上大多以 `cnts.chunks(2).map(((qst, ans)) => fn(qst, ans)).flatten()` 为基础;
 - 后续处理: 将问题题干变为问题 (`sxj-qg-to-qst-zh`), 将问题与相应答案打包 (`sxj-qg-pack`, 方便排版的同时确保答案与问题 (在 `metadata` 中的信息³) 匹配) .

② 用于批量处理的具体函数: 命名均包含 `sxj-qg` (且不含 `bchr`), 此处不作详细介绍.

③ 常用的批量处理函数数组: 命名均包含 `sxj-qg-bchr`, 此处不作详细介绍.

(2) `reference.typ`

① `sxj-ref-to-question`: 至问题的引用, 默认显示与当前题号不同的部分.

(3) `utils.typ`

① `sxj-student-info`: 学生信息栏, 默认显示“班级, 姓名, 学号”的空;

② `sxj-options`: 等距显示以 A、B、…… 编号的选项, 自动根据各选项宽度及 `col` 确定列数;

③ 可用于默认函数替代/非关键小工具:

- `sxj-title`: 居中加粗标题, 可用于替代 `title`;
- `sxj-equ`: 两侧添加 `spacing`, 用 `display` 模式显示公式, 可用于替代 `equation`;
- `sxj-unit`: 用于 (在式中) 显示单位, 如: `$1$#sxj-unit[元]`;
- `sxj-footer`: 用于页脚页码, 在 `page.footer` 中使用.

4. 封装导出: `lib.typ`

此文件中不应也不会为本包添加除了绑定之外的任何 (新) 功能.

(1) 提供常用函数的缩写别名 (见文件开头的一堆 `#let`)

- 方便读写;
- 方便 `hack`, 例如若不需要或极少需要更新二级标题则可:

```
#let cu = sxj-counter-question-update.with(level: 3)
以覆盖默认 cu.
```

(2) `shuxuejuan`, 建议使用 `show: shuxuejuan`, 当然也可以不用或复制部分源码选择性启用下列功能:

① 将常用函数与内置函数绑定:

- 将 `sxj-question-zh` 绑定到 `heading`;
- 将 `sxj-ref-to-question` 绑定到 `ref`;
- 将 `sxj-title` 绑定到 `title`;
- 将 `sxj-equ` 绑定到 `math.equation`.

② 进行默认设置:

- 句号显示为句点;
- 页面设置为 16 开⁴, 缩小页边距, 设置页码页脚.

⁴ 此处指 18.4 cm × 26 cm, 根据本人经验, 各处的 16 开大小不一, 你可能需要根据实际情况调整.

五、关键概念详解（Q&As）

1.counter-with-acc是啥，为什么需要它？

在实际出卷时，我们常用连续的二级标题：如：一、→1.→2.→二、→3.（注意最后一个编号是3.而不是重新从1.开始）。为优雅解决此类问题（使各级标题可使用连续/不连续的编号），我们约定counter-question（一个array）结构如下（下表中的“计数器”均指counter-question，“标题”亦指问题）：

- 第一个数：总计数，只要该计数器更新就自增1，可作为id使用；
- 第二个数：当前（最后更新）的标题层级；
- 第三个及之后的数，两个一组，对应各级的两种计数：
 - 各组第一个数：该级的累计计数，只要该级更新，就自增1；
 - 各组第二个数：该级的（一般）计数，只要上级⁵标题更新就从0开始计数。

例如在经过一、→1.→2.→二、→3.后counter-question将变为：

5	2	2	2	3	1
总计数	当前层级	一级累计计数	一级计数	二级相关计数（两个数含义同前）	

注意计数器（.len()）不会变短，以确保各级的累计计数能被保留，故第二个数一方面可帮助快速确定当前计数器至第几位是有用的，另一方面可简化代码（可直接用.chuncks(2)来获取各级计数）。

如此设置的counter在本项目中被称为counter-with-acc，相关代码命名中均含counter-with-acc。

2.counter-question是如何转化为显示出来的数的？

- (1)先通过env中的fn-number去除各级中不需要的累计计数/普通计数，各级只留一个数：如(5, 2, 2, 2, 3, 1)，可通过内置（常用）的
- sxj-counter-with-acc-to-nums-default转化为(2, 3)（下面以此为例）；
 - 或sxj-counter-with-acc-to-nums-normal转化为(2, 1)。
- (2)通过sxj-numbering-numbers将各级编号转化为用以显示的文字，如(2, 3)会被转化为("二、", "3.")；
- (3)根据实际需要得到一个用以显示的字符串，如对("二、", "3.")：
- 在sxj-question中，我们通过nums-to-num获取当前层级的那个（未必是最后一个，详见 1.）"3."；
 - 在sxj-ref-to-question中，我们根据引用处所在问题层级，可能得到"二、3."或"3."。

3.为什么需要sxj-term（与sxj-equ）：

在本包中，我们希望以右图方式排版题目（灰线作对齐参考），这看起 题号、竹杖芒鞋轻胜马，谁怕？来用默认的par, term, grid可以完成，但实际需考虑以下情况：

- 在中小学的练习与试卷排版中，无论是否在行内，数学公式总是以display而非默认的inline模式显示的，即用“ $\frac{1}{2}$ ”而非“ $\frac{1}{2}$ ”。“较高”的公式出现在题干首行则很可能导致题干首行与题号上下不对齐。
- 对于含图表等的内容，我们可能需要用grid来进行更精细的排版设置。若我们的body就是grid或是一段文字后再用#grid则很可能出现自动但不必要的换行。

经个人测试，用 Typst 提供的par, term, grid中的任一无法解决上述两个问题，于是将三者⁶综合为sxj-term以提供同一接口，搭配composer来指定实际的实现，sxj-get-composer-for尽可能实现自动选择composer。

在Typst 0.15.0中该问题被部分解决，但目前grid仍无法达到sxj-get-composer-for的效果。以下示例显示了grid的不足：

```
=  $\sqrt{4}$ ,  $\sqrt[3]{9}$ ,  $\frac{11}{3}$ ,  $\frac{\pi}{2}$ , 3.14, 0.301300130001dots$ ($3$与$1$间依次增加一个$0$) 中的有理数有 $\sqrt{4}$ ,  $\frac{11}{3}$ , 3.14$].

=  $\sqrt{81}$ 的平方根是 $\pm 3$ ,  $\sqrt[3]{9}$ 的立方是9,  $\sqrt{0,1}$ 的算术平方根是它本身.

=  $\sqrt{x+2}$ 与 $\sqrt{y-4}$ 互为相反数, 则 $x, y$ 的立方根是 $-2$ .

= 已知 $\sqrt[3]{15} \approx 2.466$ ,  $\sqrt[3]{150} \approx 5.313$ , 则 $\sqrt[3]{15} - 0.015 \approx -0.2466$ .

// == 整数 $m$ 满足 $m \leq \sqrt{6}(2 - \sqrt{6})\sqrt{6} < m+1$ , 则 $m = -2$ .
= 整数 $m$ 满足 $m \leq 1 - \sqrt{11} < m+1$ , 则 $m = -3$ .

= 若 $2m-1, m, 4-m$ 这三个实数在数轴上对应的点从左到右依次排列, 则 $m$ 的取值范围是 $m < 1$ .

= 统计本班同学身高时, 发现50个数据中身高最小值为150cm, 最大值为176cm, 若想体现本班同学身高分布情况, 应选用[直方]图, 若绘图时选取组距为4cm, 则应分7组.

=  $\begin{cases} x+2y=3-3k, \\ x-3y=7k-2 \end{cases}$ 是一个关于 $x, y$ 的二元方程组, 若它的解满足 $x, y$ 互为相反数, 则 $k = -2$ ; \ 若它的解满足 $3x+y>4$ , 则 $k$ 的范围是:  $k>0$ .
//  $\begin{cases} x=1+k, \\ y=1-2k \end{cases}$ 
```

⁵当然“一级”无上级，此处特指“二级”及以下的标题如此计数。
⁶按本人经验term或grid实现不了的par也不行，par仅留着以防万一。

六、Known issues & Help wanted

1. 复杂问题的题干与标号对齐，如题干为

`grid(columns: (auto, 1fr), [竹杖芒鞋轻胜马$1/2$], table([这是一个表]))`

时的排版。

当前建议使用`v(-.5em)`等微调。可考虑用`box`套住`equation`，但这很可能引入别的问题。

2. 跨页保持问题缩进。

题干不过长则无影响。

3. (在一个地方?) 多次使用`env-upd`会导致`convergence issue`。

那就不要这么用`env-upd`，如：使用

```
#context env-upd(  
  font-size: env-get("font-size") + (medium: 11pt),  
  fn-number: sxj-counter-with-acc-to-nums-normal,  
  ans-shown: false,  
)
```

代替

```
#context env-upd(font-size: env-get("font-size") + (medium: 11pt))  
#context env-upd(fn-number: sxj-counter-with-acc-to-nums-normal)  
#context env-upd(ans-shown: false)
```

可大幅减少此问题发生概率。

4. 绑定内置函数与不绑定（使用/不使用`show: shuxuejuan`）时问题行间距可能不一致。

不混用则问题不大。

5. 充满问题的`sxj-qg-add-auto-punc`，在`sxj-question-group`中使用该函数可能触发：

(1) 警告：“layout did not converge within 5 attempts”。

已知触发条件（同时满足）：

- 给`sxj-question`传入的`hanging-indent`非`length`（即`sxj-question`内调用`measure`）；
- 在`sxj-question-group`中调用`sxj-qg-add-auto-punc`功能，恰生成了当前层级的最后一个问题（即判断并生成`[.]`作为最后一个问题结尾标点）。

故当前建议`env`中的`qst-tag-w`（见 四、1.(1)②）以`auto`与`length`类型值为元素以避免该问题。

(2) 错误：“array index out of bounds”。

已知触发条件：整篇文章仅使用了本包中的一个`sxj-question-group`，则其中参数`level`的任意值几乎都会触发该错误。

当然，不使用`sxj-qg-add-auto-punc`则完全不用考虑该问题。

当然搞不懂怎样获取正确地在使用该函数时获取题号并去除警告还是很烦，so... help wanted.

6. 当前示例文件仍不够好（见 三、 ）。

我需要：

- 本身就好，能体现数学思想；
- 能体现本包的排版控制与特点（见 一、3.）；
- 可以略长，但不要过长（我讨厌在数学题里做阅读题）。

的数学题，但我现在没时间精力想了:(